

Kotlin 数组教学

Kotlin数组教学讲义（零基础入门）

各位同学，今天咱们专门学习Kotlin中的**数组（Array）**。数组就像一个“有序的储物柜”，能按顺序存放多个同类型的数据（比如一串数字、一组名字），方便我们批量管理和操作。数组在开发中非常常用，比如展示列表数据、处理批量信息等。这节课用“生活例子+代码实操”的方式，保证小白也能轻松掌握！

一、什么是数组？

想象一个“抽屉柜”，每个抽屉有固定位置（编号从0开始），每个抽屉里只能放同一种东西（比如全是书本，或全是文具）——这就是数组：

- **有序**：每个元素有固定位置（索引，从0开始）；
- **同类型**：只能存相同类型的数据（比如全是整数，或全是字符串）；
- **长度固定**：创建时确定大小，之后不能改变（想变？可以转成列表哦）。

二、数组的创建：3种常用方式

1. 用 `arrayOf()` 创建（最常用）

直接传入元素值，Kotlin会自动推断数组类型。

Code block

```
1 fun main() {  
2     // 1. 字符串数组（存名字）  
3     val names = arrayOf("小明", "小红", "小刚")  
4     println("名字数组: ${names.contentToString()}") // 输出: [小明, 小红, 小刚]
```

```

5
6    // 2. 整数数组（存成绩）
7    val scores = arrayOf(90, 85, 95, 78)
8    println("成绩数组: ${scores.contentToString()}") // 输出: [90, 85, 95, 78]
9
10   // 3. 混合类型？不推荐！（数组最好存同类型，否则失去意义）
11   val mix = arrayOf(1, "苹果", true) // 类型会被推断为Array<Any>
12 }

```

注意：打印数组不能直接 `println(names)`，会显示地址，要用 `contentToString()` 才能看到元素。

2. 用 `Array` 构造函数创建（指定长度和初始化规则）

适合需要“按规律生成元素”的场景（比如0到9的数字）。

Code block

```

1  fun main() {
2      // 创建长度为5的数组，每个元素是索引*2 (0,2,4,6,8)
3      val evenNumbers = Array(5) { index -> index * 2 }
4      println("偶数数组: ${evenNumbers.contentToString()}") // 输出: [0, 2, 4, 6,
5          8]
6
7      // 创建长度为3的数组，每个元素是"用户+索引" (用户0, 用户1, 用户2)
8      val users = Array(3) { "用户$it" } // it是默认参数名，代表当前索引
9      println("用户数组: ${users.contentToString()}") // 输出: [用户0, 用户1, 用户2]
10 }

```

3. 基本类型专用数组（更高效）

Kotlin为 `Int`、`Double` 等基本类型提供了专用数组（如 `IntArray`、`DoubleArray`），比普通 `Array` 更省内存，操作更快。

```

1 fun main() {
2     // 整数专用数组 (IntArray)
3     val ages = intArrayOf(18, 20, 22) // 等价于Array<Int>, 但更高效
4     println("年龄数组: ${ages.contentToString()}") // 输出: [18, 20, 22]
5
6     // 小数专用数组 (DoubleArray)
7     val weights = doubleArrayOf(60.5, 55.3, 70.2)
8     println("体重数组: ${weights.contentToString()}") // 输出: [60.5, 55.3, 70.2]
9
10    // 其他专用数组: booleanArrayOf(), charArrayOf()等
11 }

```

小技巧：优先用专用数组（如 `IntArray`）处理基本类型，性能更好。

三、数组的核心操作：访问、修改、遍历

1. 访问元素（通过索引）

数组的索引从0开始（第一个元素索引是0，第二个是1，以此类推）。

Code block

```

1 fun main() {
2     val fruits = arrayOf("苹果", "香蕉", "橙子")
3
4     // 访问第一个元素 (索引0)
5     println("第一个水果: ${fruits[0]}") // 输出: 苹果
6
7     // 访问最后一个元素 (索引=长度-1)
8     println("最后一个水果: ${fruits[fruits.size - 1]}") // 输出: 橙子 (size是数组长度)
9 }

```

2. 修改元素（通过索引赋值）

用 `数组名[索引] = 新值` 修改指定位置的元素（注意：数组长度不能改，但元素值可以改）。

Code block

```
1 fun main() {
2     val numbers = intArrayOf(1, 2, 3)
3     println("修改前: ${numbers.contentToString()}") // [1, 2, 3]
4
5     // 修改索引1的元素（第二个元素）
6     numbers[1] = 20
7     println("修改后: ${numbers.contentToString()}") // [1, 20, 3]
8 }
```

3. 遍历数组（常用4种方式）

遍历就是“逐个访问数组中的元素”，开发中最常用的操作之一。

Code block

```
1 fun main() {
2     val scores = intArrayOf(88, 92, 76, 95)
3
4     // 方式1: for循环遍历元素
5     println("方式1: 遍历元素")
6     for (score in scores) {
7         print("$score ") // 输出: 88 92 76 95
8     }
9     println()
10
11    // 方式2: for循环遍历索引（需要同时用索引和元素时）
12    println("方式2: 遍历索引和元素")
13    for (i in scores.indices) { // indices是数组的索引范围 (0..3)
14        println("索引$i 的元素是: ${scores[i]}")
15    }
16
17    // 方式3: forEach遍历（函数式风格，简洁）
18    println("方式3: forEach遍历")
19    scores.forEach { score ->
20        print("$score ") // 输出: 88 92 76 95
21    }
```

```
22     println()
23
24     // 方式4: 带索引的forEach
25     println("方式4: 带索引的forEach")
26     scores.forEachIndexed { index, score ->
27         println("索引$index 的元素是: $score")
28     }
29 }
```

四、数组的常用属性和方法

操作	代码示例	说明
获取长度	<code>scores.size</code>	返回数组元素个数
判断是否为空	<code>scores.isEmpty()</code>	长度为0返回true
查找元素索引	<code>scores.indexOf(92)</code>	返回第一个匹配元素的索引， 没有返回-1
检查是否包含元素	<code>92 in scores</code>	元素存在返回true
转成列表（可变）	<code>scores.toMutableList()</code>	数组转成可变列表（可增删元素）

Code block

```
1  fun main() {
2      val scores = intArrayOf(88, 92, 76, 95, 92)
3
4      println("数组长度: ${scores.size}") // 输出: 5
5      println("是否为空: ${scores.isEmpty()}") // 输出: false
6      println("92第一次出现的索引: ${scores.indexOf(92)}") // 输出: 1
7      println("是否包含76: ${76 in scores}") // 输出: true
8
9      // 数组转列表（数组长度固定，转成列表后可增删）
10     val scoreList = scores.toMutableList()
11     scoreList.add(100) // 新增元素
12     println("转成列表后: $scoreList") // 输出: [88, 92, 76, 95, 92, 100]
13 }
```

五、数组在Android中的应用场景

1. **展示列表数据**：比如RecyclerView的数据源可以是数组（转成列表后使用）；
2. **批量处理数据**：比如计算一组成绩的平均分、筛选出及格的分数；
3. **资源访问**：Android中获取字符串数组资源
(`resources.getStringArray(R.array.fruits)`)。

Code block

```
1 // 示例：计算平均分
2 fun calculateAverage(scores: IntArray): Double {
3     if (scores.isEmpty()) return 0.0
4     var sum = 0
5     for (score in scores) {
6         sum += score
7     }
8     return sum.toDouble() / scores.size
9 }
10
11 fun main() {
12     val scores = intArrayOf(80, 90, 85, 95)
13     println("平均分: ${calculateAverage(scores)}") // 输出: 87.5
14 }
```

总结与作业

核心知识点

- 数组是“有序、同类型、长度固定”的容器；
- 创建方式：`arrayOf()`（普通）、`intArrayOf()`（基本类型专用）、`Array` 构造函数；
- 核心操作：通过索引访问/修改元素，4种遍历方式；
- 常用技巧：用 `contentToString()` 打印，转成列表实现增删。

课后作业

1. 创建一个字符串数组，存放5个城市名；
2. 遍历数组，打印每个城市名和它的索引；
3. 查找“北京”在数组中的索引，如果存在则修改为“上海”；
4. 计算数组中所有城市名的平均长度（每个名字的字符数之和 $\div$ 5）。

试着自己写代码实现，有问题随时问哦！

（注：文档部分内容可能由 AI 生成）